

Trabalho 3 de Estrutura de Dados

Elainne Rohs

Joana Fontes

18 de maio de 2026

1 Algoritmos de ordenação

Usa-se os algoritmos de ordenação para transformar a fila **players** em fila de prioridade, para poder facilitar e diminuir o custo da operação **formGroup**.

O algoritmo de Insertion Sort foi implementado com apoio da função **goesFirst**, que verifica qual dentre dois jogadores possui maior prioridade. Ele percorre o vetor a partir da segunda posição e insere cada elemento na posição correta entre os anteriores já ordenados. Já o Merge Sort foi implementado pela abordagem de divisão e conquista, dividindo recursivamente o vetor em duas partes até subproblemas de tamanho 1, para então realizar a intercalação ordenada.

Recursão no Merge Sort

A cada chamada do método **mergeSort** são realizadas duas chamadas recursivas: uma para a primeira metade da fila ou de uma subfila a ser ordenada e outra para a ordenação da outra metade. Assim, quando chegar em **n=size** casos de sublistas, finalmente executa-se a função **merge** que une cada par subfilas de tamanho 1 até $n/2$. A profundidade da recursão é aproximadamente $\log_2(n)$, e cada nível percorre todos os elementos, resultando em custo $O(n \log n)$.

Aproveitamento da ordenação para formar grupos

Dada a fila **players** ordenada e a quantidade de membros requerida para o grupo a ser formado (**groupSize**), não é necessário avaliar todas as combinações possíveis de jogadores em espera existentes até achar uma que satisfaça às condições.

Basta, na verdade, considerar as combinações formadas pelos jogadores desde a posição i à posição $i + \text{groupSize} - 1$ da fila ordenada e avaliar se a diferença de scores entre o primeiro e último jogadores dessa fila é menor que o **delta**, sendo que i vai do 0 ao tamanho da fila menos o tamanho do grupo ($\text{size} - \text{groupSize}$).

Ou seja, como a lista está ordenada em ordem crescente de scores, em vez de testar $\binom{\text{size}}{\text{groupSize}}$ combinações, testa-se apenas $\text{size} - \text{groupSize} + 1$ combinações.

2 Custos Computacionais

2.1 Inserção

Basta adicionar ao array **players** o jogador, caso a memória do array não esteja totalmente ocupada $O(1)$ e somar 1 à variável **size**. Custo: $O(1)$.

2.2 Remoção

1. Percorre o array **players** até encontrar algum jogador que possua o **id** procurado. Pior caso: $O(n)$.
2. Caso esse jogador exista, cada um dos jogadores que estava atrás dele na fila ocupa a posição anterior, eliminando esse. $O(n)$.

Custo final: $O(n)$.

2.3 Função goesFirst

Apenas avalia se um jogador passa à frente do outro através da comparação de score e, talvez, timestamp.

Custo: $O(1)$

2.4 Insertion Sort

Analisando no código do corpo da função:

```
1. for(int i = 1; i < size; i++)
2. while(j>=0 goesFirst(key, players[j])) // i-1 = 0, 1, ..., n-2.
```

Portanto, o número total de operações de comparação ou troca ($T(n)$) é a soma do número de passos do loop interno para cada iteração do loop externo:

$$T(n) = 1 + 2 + 3 + \dots + (n - 1) = (n - 1)n/2$$

Custo final: $O(n^2)$.

2.5 Merge Sort

A cada chamada da função `mergeSort` são realizadas duas chamadas recursivas. Assim: $2T(n/2)$.

Ao fazer a união das duas subfilas ordenadas, pela função `merge`, são percorridos todos os elementos, então o custo é linear (cn). Assim, analisando a recorrência: $T(n) = 2T(n/2) + cn$

Como mostrado pela árvore de recursão:

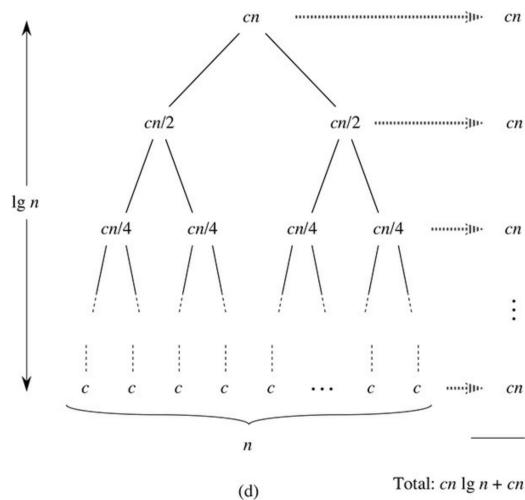


Figura 2.5 Como construir uma árvore de recursão para a recorrência $T(n) = 2T(n/2) + cn$. A parte (a) mostra $T(n)$, que expande-se progressivamente em (b)-(d) para formar a árvore de recursão. A árvore completamente expandida da parte (d) tem $\lg n + 1$ níveis (isto é, tem altura $\lg n$, como indicado) e cada nível contribui com o custo total cn . Então, o custo total é $cn \lg n + cn$, que é $\Theta(n \lg n)$.

No início, temos um problema com custo de `merge` de cn , ou seja, custo total de cn . No nível abaixo, são dois problemas cada um com custo $c(n/2)$ na união, também com custo total de cn .

E assim sucessivamente, até ter, no nível $\log_2 n$, n problemas, cada um com custo c .

Como cada nível custa cn e são $\log_2 n$ níveis, o custo é $T(n) = cn \log_2 n + cn$. E portanto:

Custo final: $O(n \cdot \log(n))$.

2.6 Formação de grupo

Como explicado na subseção *Aproveitamento da ordenação para formar grupos*, testa-se apenas $size - groupSize + 1$ combinações ou menos, até achar um grupo que atenda à condição do delta. Assim, no pior caso:

Custo: $O(n)$.

2.7 Recuperação dos jogadores

Após a criação de um array dinâmico vazio, percorre-se cada elemento do array estático `players` e copia-se no array dinâmico.

Custo: $O(n)$

Operação	Complexidade	Justificativa
Inserção	$O(1)$	Inserção direta no final do vetor
Remoção	$O(n)$	Busca linear e deslocamento
Insertion Sort	$O(n^2)$	Comparações sucessivas
Merge Sort	$O(n \log n)$	Divisão recursiva e merge
Formação de grupo	$O(n)$	Janela deslizante
Recuperação dos jogadores	$O(n)$	Cópia do vetor

3 Insertion sort vs Merge sort

Realização de testes

Foram realizados quatro testes com entradas de 1000, 3000, 5000 e 10000 jogadores. Os tempos foram medidos com `high_resolution_clock` da biblioteca `chrono`. Para garantir comparação justa, os dois algoritmos receberam exatamente a mesma sequência de jogadores.

3.1 Resultados

Os tempos resultantes (em milissegundos) foram:

n	n^2	$n \log_2 n$
1000	10^6	$\approx 10^4$
3000	$9 \cdot 10^6$	$\approx 3,5 \cdot 10^4$
5000	$25 \cdot 10^6$	$\approx 6,15 \cdot 10^4$
10000	$100 \cdot 10^6$	$\approx 13,3 \cdot 10^4$

Tamanho	Insertion (ms)	Merge (ms)
1000	10	0
3000	43	1
5000	98	2
10000	411	4

Esses resultados ratificam o custo computacional de ambos algoritmos de ordenação. É perceptível que os tempos dos testes de Merge Sort possuem um crescimento mais suave que os de Insertion Sort, $n \log n$ cresce mais lentamente que n^2 .

Os tempos reais não são exatamente proporcionais a n^2 no caso de Insertion Sort e a $n \log n$ no caso de Merge Sort porque existem otimizações do compilador, clock da máquina, cache, entre outros fatores que influenciam a eficiência dos algoritmos. O importante é a tendência de crescimento. Mesmo assim, nos casos $n = 5\text{mil}$ e 10 mil , $100 \cdot 10^6 / 25 \cdot 10^6 = 4$ e $411\text{ms} / 98\text{ms} \approx 4$.

4 Conclusão

A implementação do sistema demonstrou como a escolha do algoritmo de ordenação impacta diretamente a eficiência do matchmaking. A utilização da ordenação permitiu simplificar a formação de grupos e reduzir custos de busca. Experimentalmente, o Merge Sort mostrou-se superior ao Insertion Sort para entradas maiores, sendo a solução recomendada para cenários em que a fila de jogadores pode crescer significativamente.

5 Link do Repositório

<https://github.com/earohs/Lista-3---Joana-Fontes-e-Elainne-Rohs---Estrutura-de-Dados->